

LAB 6

MIPS INSTRUCTIONS

CREATED BY ENG: ABD ELSALAM SAEED

How to Getting User's Input in MIPS?

In MIPS assembly language, getting user input involves using system calls(via the syscall instruction).

Here's how to handle different types of user input in MIPS

Syscall Code	Function	Registers Involved
5	Read integer	Result in \$v0
6	Read float	Result in \$f0
8	Read string	\$a0 = buffer, \$a1 = max length

Getting user input integers

To read an integer from the user:

- Load the syscall code 5 into \$v0.
- Use syscall.
- The integer input will be stored in \$v0

Example Code for Integer Input

.data

- input: .word 0 # Reserve memory to store the integer input

.text

- li \$v0, 5 # Syscall code for reading an integer
- syscall # Perform the syscall
- sw \$v0, input # Store the input in memory

Getting user input floats

To read a floating-point number:

- Load the syscall code 6 into \$v0.
- Use syscall.
- The floating-point input will be stored in \$f0

Example Code for Float Input

.data

- input_float: .float 0.0 # Reserve memory for float input

.text

- li \$v0, 6 # Syscall code for reading a float
- syscall # Perform the syscall
- swc1 \$f0, input_float # Store the float input in memory

Getting user input String

To read a string from the user:

- Load the syscall code 8 into \$v0.
- Use \$a0 to provide the buffer address and \$a1 for the maximum input length.
- Use syscall.
- The string will be stored in the buffer.

Example Code for String Input

.data

- buffer: .space 50 # Allocate space for the string (50 bytes)
- prompt: .ascii "Enter a string: "

.text

Print the prompt

- li \$v0, 4 # Syscall code for print_string
- la \$a0, prompt # Load the address of the prompt
- syscall # Perform the syscall

Get the string input

- li \$v0, 8 # Syscall code for reading a string
- la \$a0, buffer # Load the address of the buffer
- li \$a1, 50 # Set the maximum length of the input
- syscall # Perform the syscall

LABELS IN MIPS

In MIPS assembly, **labels** are used to mark specific points in the program, usually to define locations for jumps, branches, or data declarations. They help make code more readable and allow efficient navigation in your program.

Syntax of Labels

A label is a name followed by a colon (:).

It represents the address of the instruction or data that follows it.

Example:

- label_name:
 - # Instructions or data follow

Usage of Labels

1. Labels in Instructions :-

- Labels are commonly used to identify targets for branching and jumping instructions, such as j, beq, and bne.

2. Labels for Loops

- Labels are also useful in implementing loops.

3. 3. Labels in Data Segments

- Labels in the .data section are used to declare memory locations for variables.

Best Practices for Using Labels

Use Descriptive Names:

- Labels like main, loop, or exit are intuitive. Avoid single-character or unclear names.

Avoid Label Conflicts:

- Each label in a program must be unique.

Keep Labels Aligned with Purpose:

- Place labels directly above the code or data they represent.

If Statements Branching Instructions

In MIPS assembly, if statements are implemented using branching instructions.

These instructions control the flow of the program based on the result of comparisons.

Common branching instructions include beq, bne, and conditional set instructions like slt

Branching Instructions for If Statements

beq (Branch if Equal)

- Branches to a label if two registers are equal.
- Syntax:
 - beq \$rs, \$rt, label

bne (Branch if Not Equal)

- Branches to a label if two registers are not equal.
- Syntax:
 - bne \$rs, \$rt, label3.

slt (Set on Less Than)

- Compares two registers and sets a destination register to 1 if the first is less than the second, otherwise sets it to 0. Often combined with beq or bne for conditional branching.
- Syntax:
 - slt \$rd, \$rs, \$rt

Example: Simple If Statement

The equivalent of this C code:

```
if (a == b)
{
    c = 1;
}
else
{
    c = 0;
}
```

MIPS Code

.data

- a: .word 5
- b: .word 5
- c: .word 0

.text

main:

- lw \$t0, a # Load a into \$t0
- lw \$t1, b # Load b into \$t1
-
- # Compare a and b
- beq \$t0, \$t1, equal # If a == b, jump to 'equal'
- li \$t2, 0 # Else, set c = 0
- sw \$t2, c
- j end # Jump to the end
- equal:
- li \$t2, 1 # Set c = 1
- sw \$t2, c
- end:
- # Exit program
- li \$v0, 10
- syscall

Example: If-Else with Relational Operators

The equivalent of this C code:

```
if (a < b)
{
    c = 1;
}
else
{
    c = 0;
}
```

MIPS Code

.data

- a: .word 3
- b: .word 5
- c: .word 0

.text

main:

- lw \$t0, a # Load a into \$t0
- lw \$t1, b # Load b into \$t1

Check if a < b

- slt \$t2, \$t0, \$t1 # \$t2 = 1 if \$t0 < \$t1, else \$t2 = 0
- bne \$t2, \$zero, less # If \$t2 != 0 (a < b), branch to 'less'
- li \$t3, 0 # Else, set c = 0
- sw \$t3, c
- j end # Jump to the end

less:

- li \$t3, 1 # Set c = 1
- sw \$t3, c

end:

Exit program

- li \$v0, 10
- syscall

Summary

1

Use beq or bne for equality/inequality checks.

2

Use slt for less-than comparisons. Combine with bne or beq to create branches.

3

Always ensure you properly set up the labels and handle the program flow for both the true and false conditions.

Branching pseudo instructions

In MIPS assembly language, pseudo-instructions are simplified, high-level instructions that the assembler translates into one or more actual hardware-supported instructions (real instructions). For branching, pseudo-instructions are often used for ease of programming.

Common Branching Pseudo-Instructions

b (Branch)

- A simple branch to a label without any condition.
- Example:
 - b label
- Equivalent Real Instruction:
 - j label # Jump unconditionally to the label

blt (Branch if Less Than)

- Branches to a label if one register is less than another.
- Example:
 - blt \$t0, \$t1, label
- Equivalent Real Instructions:
 - slt \$at, \$t0, \$t1 # \$at = 1 if \$t0 < \$t1
 - bne \$at, \$zero, label

Common Branching Pseudo-Instructions

ble (Branch if Less Than or Equal)

- Branches to a label if one register is less than or equal to another.
- Example:
 - ble \$t0, \$t1, label
- Equivalent Real Instructions:
 - slt \$at, \$t1, \$t0 # \$at = 1 if \$t1 < \$t0
 - beq \$at, \$zero, label

bge (Branch if Greater Than or Equal)

- Branches to a label if one register is greater than or equal to another.
- Example:
 - bge \$t0, \$t1, label
- Equivalent Real Instructions:
 - slt \$at, \$t0, \$t1 # \$at = 1 if \$t0 < \$t1
 - beq \$at, \$zero, label

Common Branching Pseudo-Instructions

bnez (Branch if Not Zero)

- Branches to a label if a register is not zero.
- Example:
 - bnez \$t0, label
- Equivalent Real Instruction:
 - bne \$t0, \$zero, label

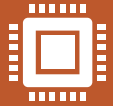
beqz (Branch if Equal to Zero)

- Branches to a label if a register is zero.
- Example:
 - beqz \$t0, label
- Equivalent Real Instruction:
 - beq \$t0, \$zero, label

bgt (Branch if Greater Than)

- Branches to a label if one register is greater than another.
- Example:
 - bgt \$t0, \$t1, label
- Equivalent Real Instructions:
 - slt \$at, \$t1, \$t0 # \$at = 1 if \$t1 < \$t0 (or \$t0 > \$t1)
 - bne \$at, \$zero, label

Benefits of Pseudo-Instructions



They make assembly programming easier and more intuitive.



They abstract away the need to manually handle intermediate comparisons using registers like \$at (assembler temporary).



The assembler takes care of translating pseudo-instructions into actual MIPS instructions.

While loop in MIPS

A while loop in MIPS assembly is implemented using a combination of labels and branching instructions.

The structure mimics a C-like while loop:

C

```
◦ while (condition)
    {
        // body
    }
```

In MIPS, you check the condition at the start of the loop and branch out if it evaluates to false. Here's how it works:

General Structure of a While Loop in MIPS

General Structure of a While Loop in MIPS

loop_start:

- # Evaluate condition
- # If condition is false, branch to loop_end
- # Loop body
- j loop_start # Jump back to the start of the loop

loop_end:

- # Continue program

Example: Counting Down with a While Loop

This example sums numbers from 1 to 5. C Code:

C

- `int sum = 0;`
- `int i = 1;`
- `while (i <= 5)`
 - `{`
 - `sum += i;`
 - `i++;`
 - `}`

Example: Counting Down with a While Loop

MIPS Assembly:

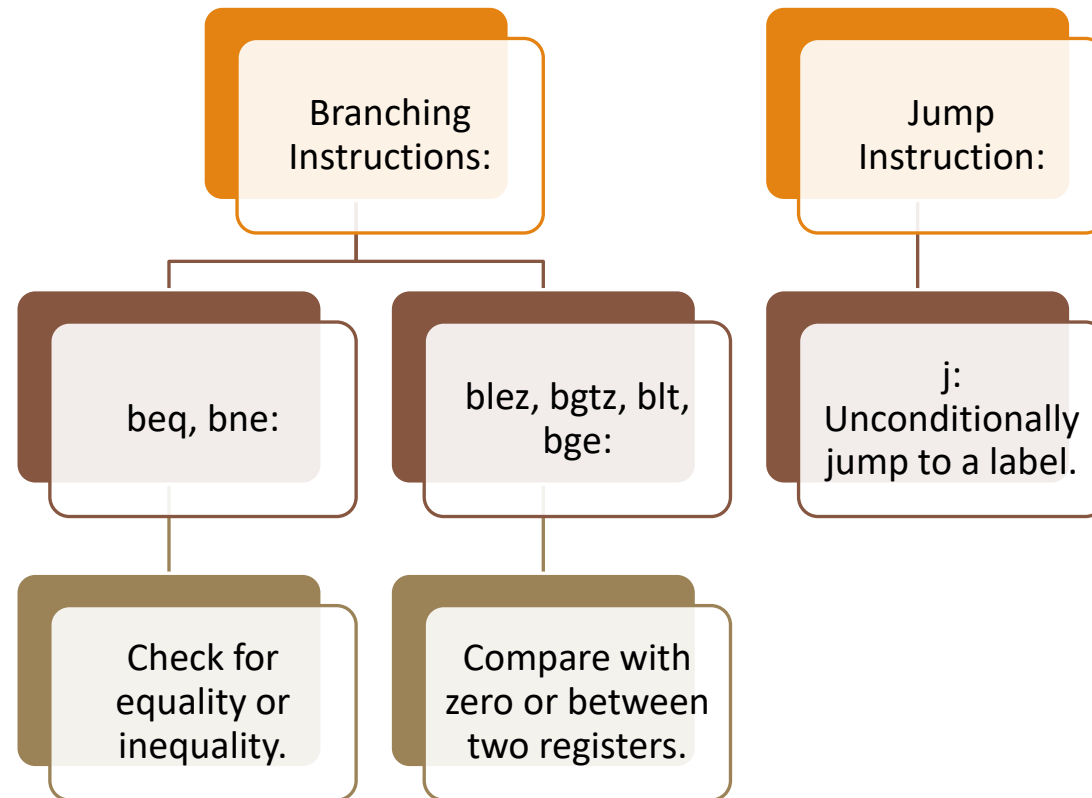
.data

- sum: .word 0 # Variable to store the sum
- i: .word 1 # Loop counter
- n: .word 5 # Upper limit

.text

- main:
 - lw \$t0, i # Load i into \$t0
 - lw \$t1, n # Load upper limit into \$t1
 - lw \$t2, sum # Load sum into \$t2
- loop_start:
 - bgt \$t0, \$t1, loop_end # If i > n, exit loop
 - add \$t2, \$t2, \$t0 # sum += i
 - addi \$t0, \$t0, 1 # i++
 - j loop_start # Jump back to the start of the loop
- loop_end:
 - sw \$t2, sum # Store updated sum
 - li \$v0, 10 # Exit syscall
 - syscall

Key Instructions for While Loops



How the While Loop Works in MIPS

Condition Evaluation:

- The loop condition is checked using comparison instructions like blez (branch if less than or equal to zero).

Loop Body:

- If the condition is true, the loop body executes.

Iteration Control:

- After executing the body, a jump (j) returns control to the condition evaluation.

Exit:

- When the condition becomes false, the program branches to the label outside the loop.

Arrays In MIPS

In MIPS assembly language, arrays are represented as blocks of contiguous memory in the .data section, and you access individual elements using their memory address and an offset.

Since MIPS does not have a dedicated array data structure, you manage arrays manually with memory and registers.

Declaring an Array

Arrays are declared in the .data section

- using directives like
 - .word,
 - .byte, or
 - .float.

Integer Array Example:

- .data
 - arr: .word 10, 20, 30, 40, 50 # Declare an array of integers
 - size: .word 5 # Size of the array

Floating-Point Array Example:

- .data
 - f_arr: .float 1.1, 2.2, 3.3, 4.4 # Declare an array of floats

Accessing Array Elements

To access elements of an array:

1. Load the base address of the array into a register using the `la` (load address) instruction.
2. Use an offset to access individual elements.
3. The offset depends on the size of the data type:
 - Integers (`.word`): 4 bytes per element
 - Bytes (`.byte`): 1 byte per element
 - Floats (`.float`): 4 bytes per element

Accessing an Integer Array Element

Accessing an Integer Array Element:

Access the 3rd element (index 2, since arrays are zero indexed):

.text

- main:
 - la \$t0, arr # Load base address of 'arr' into \$t0
 - li \$t1, 2 # Index = 2
 - sll \$t1, \$t1, 2 # Multiply index by 4 (size of .word)
 - add \$t2, \$t0, \$t1 # \$t2 = address of arr[2]
 - lw \$t3, 0(\$t2) # Load arr[2] into \$t3

Accessing a Floating-Point Array Element

Access the 2nd element (index 1):

.text

- main:
 - la \$t0, f_arr # Load base address of 'f_arr'
 - li \$t1, 1 # Index = 1
 - sll \$t1, \$t1, 2 # Multiply index by 4 (size of .float)
 - add \$t2, \$t0, \$t1 # \$t2 = address of f_arr[1]
 - lwc1 \$f1, 0(\$t2) # Load f_arr[1] into \$f1

Printing an Array with a While Loop

Array Setup:

- We declare an array in the .data section and iterate through its elements in the .text section.

Example Code:

- .data
 - arr: .word 10, 20, 30, 40, 50 # Array of integers
 - size: .word 5 # Number of elements in the array
 - newline: .ascii "\n" # Newline for formatting
- .text
 - main:
 - # Step 1: Load the base address of the array and the size
 - la \$t0, arr # Load base address of the array into \$t0
 - lw \$t1, size # Load the size of the array into \$t1
 - li \$t2, 0 # Initialize index i = 0

Printing an Array with a While Loop

loop_start:

Step 2: Check if index $i \geq \text{size}$

- bge \$t2, \$t1, loop_end # If $i \geq \text{size}$, exit the loop

Step 3: Access arr[i]

- sll \$t3, \$t2, 2 # Multiply index by 4 (word size)
- add \$t4, \$t0, \$t3 # Address of arr[i]
- lw \$a0, 0(\$t4) # Load arr[i] into \$a0

Step 4: Print the current array element

- li \$v0, 1 # Syscall code for print integer
- syscall

Printing an Array with a While Loop (Continue)

Step 4: Print the current array element (Continue)

- # Print a newline for formatting
 - la \$a0, newline # Load address of newline string
 - li \$v0, 4 # Syscall code for print string
 - syscall

Step 5: Increment index i and repeat

- addi \$t2, \$t2, 1 # i++
- j loop_start # Jump back to start of the loop

loop_end:

Step 6: Exit the program

- li \$v0, 10 # Exit syscall
- syscall

Explanation of Key Steps

1. Initialization:

- `la $t0, arr`: Load the base address of the array.
- `lw $t1, size`: Load the size of the array into a register.
- `li $t2, 0`: Initialize the index (i) to 0.

2. Condition Check:

- `bge $t2, $t1, loop_end`: Compare the index with the array size and exit the loop if the condition is met.

3. Accessing Array Elements:

- Calculate the address of the current element using the formula:
 - $\text{address} = \text{base address} + (\text{index} \times \text{element size})$
- Multiply the index by 4 (size of a word) and add it to the base address.

Explanation of Key Steps

Printing the Element:

- Load the element into `$a0` and use `syscall` to print it.

Formatting (Optional):

- Print a newline after each element to improve readability.

Incrementing and Repeating:

- Increment the index (`addi $t2, $t2, 1`) and jump back to the start of the loop.

Exiting the Program:

- Use `li $v0, 10` and `syscall` to terminate execution.